Modul Informations– und Kodierungstheorie

Aufgaben zu Lektion 1 — Lösungshinweise

Aufgabe 1-1: Morse-Code

Wir bringen den internationalen Morse-Code (s. Abb. 1.26 des Basistextes) ins Digital-Zeitalter, indem wir den Punkt als 0 und den Strich als 1 kodieren.

a) Wie viele Bits brauchen wir im Mittel bei längeren Texten, wenn die 26 Buchstaben mit relativen Häufigkeiten wie in der ersten Tabelle in

`https://de.wikipedia.org/wiki/Buchstabenh%C3%A4ufigkeit`

vorkommen (ß wird ignoriert), insgesamt 99,69% der Zeichen ausmachen, und jede Ziffer eine Häufigkeit von 0,031% hat?

Hinweis: Zur Bildung des Mittelwerts gewichtet man (=multipliziert) die Bitanzahl für jedes Zeichen mit der Häufigkeit des Zeichens, und summiert über alle Zeichen.

b) Braucht der Morse-Code damit mehr oder weniger Bits pro Zeichen als ein ASCII-Code (s. Tab. 3.3 des Basistextes), der 7 Bit pro Zeichen benötigt — der zugegebenermaßen aber auch mehr Zeichen kodieren kann?

c) Wir haben beim Morse-Code bisher die Bedeutung der Pause zwischen den Signalen verschiedener Zeichen ignoriert. Geben Sie ein Beispiel an für zwei Folgen aus zwei Zeichen, die beim gerade vorgestellten „Digital-Morse-Code“ zu gleichen Bitfolgen führen, d.h. nicht mehr eindeutig zurück verwandelt werden können.

d) Mit welchem Bitmuster könnte man die Pause kodieren, damit eine eindeutige Kodierung möglich wird? Dabei darf das Pausen-Bitmuster vom vorausgehenden Zeichen abhängen. Wird die Aufgabe einfacher, wenn man die Pause zusammen mit dem jeweils nächsten Zeichen kodiert, d.h. das Bitmuster für die Pause ein Präfix des folgenden Zeichens wird und vom dem folgenden Zeichen abhängen darf? Dabei soll angenommen werden, dass es dann auch vor dem ersten Zeichen eine Pause gibt.

Lösung

a) Es ergibt sich folgende Tabelle.

Zeichen	# Bits	Wahrsch.
A	2	6,51%
B	4	1,89%
C	4	3,06%
D	3	5,08%
E	1	17,40%
F	4	1,66%
G	3	3,01%
H	4	4,76%
I	2	7,55%
J	4	0,27%
K	3	1,21%
L	4	3,44%
M	2	2,53%
N	2	9,78%
O	3	2,51%
P	4	0,79%
Q	4	0,02%
R	3	7,00%
S	3	7,27%
T	1	6,15%
U	3	4,35%
V	4	0,67%
W	3	1,89%
X	4	0,03%
Y	4	0,04%
Z	4	1,13%
0	5	0,031%
1	5	0,031%
2	5	0,031%
3	5	0,031%
4	5	0,031%
5	5	0,031%
6	5	0,031%
7	5	0,031%
8	5	0,031%
9	5	0,031%

Damit beträgt die gewichtete Summe, d.h. die mittlere Codewort-Länge, 2,46.

b) Die Anzahl ist geringer als als beim ASCII-Code. Das ist sogar unabhängig von den Wahrscheinlichkeiten zu sehen, da kein Codewort länger als 5 Bit ist und damit die mittlere Codewort-Länge höchstens 5 Bit betragen kann.

c) Die Zeichenfolgen EI und IE werden beide, wenn die Pause ignoriert wird, zu 000 kodiert.

d) Man kann systematisch eine möglichst kurze Zeichenfolge suchen, die, wenn man sie an jeden Morsecode hängt, dazu führt, dass Präfixfreiheit eintritt. Diese Zeichenfolge muss mindestens die Länge 2 haben, denn sowohl bei Anhängen von 0 als auch bei Anhängen von 1 lassen sich einfache Gegenbeispiele finden: E ist Präfix von A bzw. I. Auch bei Anhängen von 00, 01, 10, 11, 000, 001,...,111 ist E oder T Präfix verschiedener anderer Zeichen, sogar bei 0000 bis 1111 gibt es Probleme. Also muss die Länge sogar mindestens 5 betragen. Beispielsweise führt das Anhängen von 00001 zu einem präfixfreien Code. Es sei angemerkt, dass nur ein Teil der 5-Bit-Folgen bei Anhängen zu einem präfixfreien erweiterten Morse-Code führt. Die Wahl des Suffix abhängig vom Symbol macht die Lage leider nicht einfacher, da die Anzahl der Lösungsmöglichkeiten stark ansteigt. Höchstens kann man nach dem Anhängen von zum Beispiel 00001 beginnen, die Codewörter einzelner Symbole zu „kürzen“, solange Präfixfreiheit erhalten bleibt.

Demgegenüber ist die Situation bei Voraussstellen einfach: Wir stellen einem Codewort die Länge des Codeworts (die 1 bis 5 betragen kann) minus 1 (also 0 bis 4) als 3-Bit-Binärdarstellung voran. Codewörter verschiedener Länge können dann kein Präfix des anderen sein. Codewörter gleicher Länge müssen verschieden sein, können also auch mit Voranstellen der Länge kein Präfix des anderen sein. Hier ist übrigens nur die Lösung mit Symbolspezifischem Präfix zielführend, denn bei einem gemeinsamen Präfix p für alle Codewörter würde folgen: ist Codewort a Präfix von Codewort b, dann ist auch pa Präfix von pb.

Aufgabe 1-2: Morse-Code in der Praxis

Lösen Sie Aufgabe 1.7 des Basistextes.

Lösung

a) Durch ein Leerzeichen mit gleichzeitiger Umschaltung auf die Zahlentabelle als erstem Zeichen ist sichergestellt, dass zu Anfang die Zahlentabelle genutzt wird. Stört dieses führende Leerzeichen, dann kann es mit einem darauffolgenden Löschzeichen wieder gelöscht werden, so dass das dann folgende Zeichen das eigentlich erste der Nachricht ist.

b) Hier wird zu Beginn die Methode aus a) angewendet: für das dritte und vierte Zeichen wird die Zahlentabelle genommen, also 3 und 7, dann folgt ein Leerzeichen (mit Umschaltung auf Buchstaben) und die Buchstaben G, R, A, D. Insgesamt ergibt sich die Nachricht „37 GRAD“.

Aufgabe 1-3: Zum Knobeln: Erkennung von Übertragungsfehlern

a) Sie übertragen eine digitale Nachricht Bit für Bit, und alle 8 Bit können Sie ein zusätzliches Bit übertragen, das Sie benutzen wollen, um Übertragungsfehler zu erkennen. Dabei dürfen Sie annehmen, dass in jedem Block von 9 Bits höchstens ein Bit durch einen Übertragungsfehler verfälscht wird. Geben Sie eine Formel an, wie sich das neunte Bit b_8 aus den ersten 8 Bit $b_0, \ldots, b_7$ berechnet.

b) Argumentieren Sie, weshalb diese Berechnung einen 1-Bit-Fehler erkennt.

c) Nehmen Sie nun an, dass innerhalb 3 Bits eines verfälscht werden kann. Wie viele Bits von 3 können noch von der digitalen Nachricht stammen, und wie werden die anderen, zusätzlichen Bits aus den Datenbits gebildet?

Lösung

a) Am einfachsten nimmt man die sogenannte (gerade) Parität: man zählt die Einsen in den Bits $b_0, \ldots, b_7$ und speichert, ob es eine gerade (0) oder ungerade (1) Zahl war. Hierzu reicht der Operator exklusiv-Oder:

$$b_8 = b_0 \oplus b_1 \oplus \cdots \oplus b_7 \ .$$

b) Ein Fehler in einem Bit verändert dieses von 0 zu 1 oder umgekehrt, d.h. die Parität würde in jedem Fall verändert. Damit ist jeder 1-Bit-Fehler erkennbar.

c) Selbst in diesem Fall können 2 Bits zur Datenübertragung benutzt werden und das dritte Bit bildet die Parität wie in Teil a). Bei einer korrekten Übertragung ist die Anzahl der Einsen in einem Dreierblock immer gerade. Kommt es zu einem 1-Bit-Fehler, so ist die Anzahl der Einsen ungerade, womit der Fehler stets entdeckt wird.

Wenn man die Datenübertragung auf 1 Bit pro 3er-Block reduziert, ist sogar eine Fehler*korrektur* möglich. Um eine Mehrheitsentscheidung herbeiführen zu können, verdreifacht man das einzelne Bit der digitalen Nachricht in einem solchen 3er-Block, d.h. die weiteren beiden Bits entsprechen dem Datenbit. Bei einem 1-Bit-Fehler entspricht die Mehrheit der Bitwerte dem korrekten Wert. Dies ist übrigens der einfachste *Hamming-Code*.

Aufgabe 1-4: Komprimierung von Texten

Wir gehen davon aus, dass wir in einer Bytefolge die 2^t häufigsten Byte-Paare $p_i = (b_{i,0}, b_{i,1})$ kennen, wobei $i = 0, 1, \ldots, 2^t - 1$, und die $b_{i,j} \in \{0, \ldots, 255\}$ Byte-Werte darstellen mit $j \in \{0, 1\}$. Dann können wir die Bytefolge $a = a_0, a_1, \ldots, a_{n-1}$ wie folgt komprimieren. Nehmen wir an, wir haben bereits die ersten k Bytes verarbeitet. Wenn die nächsten beiden Bytes a_k, a_{k+1} eines der häufigsten Byte-Paare bilden, z.B. p_i, dann kodieren wir sie in $t + 1$ Bit als $0, \langle i \rangle$, wobei $\langle i \rangle$ die t-Bit-Binärdarstellung der Zahl i ist, und erhöhen k um 2. Ansonsten kodieren wir das Byte a_k in 9 Bit als $1, a_k$ und erhöhen k um 1.

Wie hoch muss für $t = 7$, also $2^t = 128$ häufige Paare, der Anteil α der häufigen Paare am Gesamtumfang n sein, damit diese Kodierung (in Bit gerechnet) kürzer ist als die Länge der ursprünglichen Folge, die $8n$ Bit beträgt?

Lösung

Wir bezeichnen den Anteil der häufigsten Paare mit $\alpha \in [0; 1]$. Bei $\alpha = 1$ besteht die Bytefolge a nur aus $n/2$ häufigen Paaren. Bei $\alpha = 0.5$ besteht die Hälfte der Bytefolge (also $n/2$ Bytes) aus $n/4$ vielen häufigen Paaren, die anderen $n/2$ Bytes bestehen aus Werten, die nicht zu den häufigsten Paaren gehören. Die komprimierte Bytefolge besteht dann im Allgemeinen aus $\alpha \cdot n/2$ Paaren und $(1 - \alpha)n$ Einzelwerten, wobei jedes Paar mit 8 und jeder Einzelwert mit 9 Bit kodiert wird. Insgesamt hat die komprimierte Folge also die Länge

$$L = 8 \cdot \alpha \cdot n/2 + 9 \cdot (1 - \alpha)n = 9n - 5\alpha n$$

in Bits. Sie ist kürzer als die ursprüngliche Folge, wenn $9n - 5\alpha n < 8n$, also $\alpha > 1/5$.